

DSA Pattern Recognition Cheat Sheet

1. Two Pointers

When to use:

- 1 Sorted array or string
- 2 Need pairs/triplets
- 3 Opposite ends or slow/fast pointers

Core Idea:

Move two indices based on condition instead of nested loops.

Patterns:

- 1 Left & Right pointers
- 2 Fast & Slow pointer (cycle detection)

Example Problems:

- 1 Two Sum II
- 2 3Sum
- 3 Container With Most Water
- 4 Remove Duplicates from Sorted Array

Recognition Clue:

"Find pair", "sorted array", "optimize $O(n^2) \rightarrow O(n)$ "

2. Sliding Window

When to use:

- 1 Subarray / substring problems
- 2 Contiguous elements
- 3 Max/min length or sum

Core Idea:

Expand window $\rightarrow$ shrink when invalid.

Patterns:

- 1 Fixed window
- 2 Variable window

Example Problems:

- 1 Longest Substring Without Repeating Characters
- 2 Minimum Window Substring
- 3 Maximum Sum Subarray of Size K

Recognition Clue:

"Substring", "subarray", "continuous", "window size"

3. Binary Search

When to use:

- 1 Sorted data
- 2 Search space can be reduced
- 3 Answer lies in range

Core Idea:

Divide search space in half.

Patterns:

- 1 Classic binary search
- 2 Search on answer
- 3 Lower bound / upper bound

Example Problems:

- 1 Binary Search
- 2 Search in Rotated Sorted Array
- 3 Find Peak Element
- 4 Median of Two Sorted Arrays

Recognition Clue:

"Sorted", " $\log(n)$ ", "min/max possible value"

4. Backtracking

When to use:

- 1 Generate all combinations/permutations
- 2 Decision tree exploration

Core Idea:

Try $\rightarrow$ recurse $\rightarrow$ undo (backtrack).

Patterns:

- 1 Include / exclude
- 2 Choose / not choose

Example Problems:

- 1 Subsets
- 2 Permutations
- 3 Combination Sum
- 4 N-Queens

Recognition Clue:

"All possible", "combinations", "generate"

5. Dynamic Programming (DP)

When to use:

- 1 Overlapping subproblems
- 2 Optimal substructure

Core Idea:

Store results to avoid recomputation.

Patterns:

- 1 1D DP
- 2 2D DP
- 3 Knapsack
- 4 LIS

Example Problems:

- 1 Climbing Stairs
- 2 House Robber
- 3 Longest Common Subsequence
- 4 Coin Change

Recognition Clue:

"Min/max ways", "count ways", "optimal"

6. Greedy

When to use:

- 1 Local optimal leads to global optimal

Core Idea:

Pick best choice at each step.

Patterns:

- 1 Sorting + decision
- 2 Interval problems

Example Problems:

- 1 Activity Selection
- 2 Jump Game
- 3 Merge Intervals
- 4 Gas Station

Recognition Clue:

"Minimum steps", "maximum profit", "greedy choice works"

7. Graph (BFS / DFS)

When to use:

- 1 Nodes + edges
- 2 Traversal problems

Core Idea:

Explore neighbors.

Patterns:

- 1 BFS (queue)
- 2 DFS (recursion/stack)
- 3 Cycle detection
- 4 Topological sort

Example Problems:

- 1 Number of Islands
- 2 Clone Graph
- 3 Course Schedule
- 4 Rotten Oranges

Recognition Clue:

"Grid", "connections", "dependencies"

8. Heap / Priority Queue

When to use:

- 1 Top K elements
- 2 Min/max retrieval

Core Idea:

Maintain ordered structure.

Patterns:

- 1 Min heap
- 2 Max heap

Example Problems:

- 1 Kth Largest Element
- 2 Top K Frequent Elements
- 3 Merge K Sorted Lists

Recognition Clue:

"Top K", "smallest/largest efficiently"

9. Stack / Monotonic Stack

When to use:

- 1 Next greater/smaller element
- 2 Expression parsing

Core Idea:

Use LIFO structure.

Patterns:

- 1 Monotonic increasing/decreasing

Example Problems:

- 1 Next Greater Element
- 2 Largest Rectangle in Histogram
- 3 Valid Parentheses

Recognition Clue:

"Nearest greater/smaller", "previous/next"

10. Trie (Prefix Tree)

When to use:

- 1 Prefix-based searching
- 2 Strings

Core Idea:

Tree of characters.

Patterns:

- 1 Prefix matching

Example Problems:

- 1 Implement Trie
- 2 Word Search II
- 3 Longest Common Prefix

Recognition Clue:

"Prefix", "dictionary", "auto-complete"

Practice Strategy

- 1 Solve 3–5 problems per pattern.
- 2 After solving, note why the pattern worked.
- 3 Identify recognition clues.
- 4 Revisit mistakes weekly.